2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240881

JSA-CIM: A Joint-Sparse AdderNet Compute-In-Memory Accelerator for Energy-Efficient Edge AI Applications

Omar Al Kailani, Yunxiang Zhang, and Wenfeng Zhao
Dept. of Electrical and Computer Engineering, Binghamton University, New York, USA
{oalkail1, yzhan392, wzhao}@binghamton.edu

***Abstract*—AdderNet has emerged as a compelling alternative to the prevailing Convolutional Neural Networks, offering comparable model accuracy while significantly reducing the computational complexity of Multiply-Accumulate operations with efficient Sum-of-Absolute-Differences alternatives. Despite being promising, current Compute-in-Memory (CIM) designs for AdderNet fail to leverage the inherent model sparsity, thereby overlooking substantial opportunities for enhancing energy efficiency in CIM-based implementations. In this paper, we propose *JSA-CIM*, an 8T-SRAM-based Digital CIM (DCIM) for efficient AdderNet inference, exploiting joint-sparsity of both activations and weights in a bit-serial fashion. Specifically, JSA-CIM features (a) at architecture level, a sparsity- and kernel-size-aware CIM macro that facilitates word-level activation sparsity exploitation for energy reduction, and accommodates the memory organization to fit popular kernel sizes for maximizing the application-level throughput, (b) at circuit-level, a novel joint-sparse bitline architecture based on 8T-SRAM that can exploit both word-level activation sparsity and bit-level weight sparsity to minimize the array energy consumption, and (c) a robust and sparsity-gated minimum selection circuit to achieve reliable and energy-efficient bit-serial minimum operation. When implemented in 65nm CMOS technology, the proposed JSA-CIM demonstrates a peak throughput of 170.7 GOPS, 1.8 TOPS/mm^2 area efficiency, and 75.98 TOPS/W energy efficiency.**

***Index Terms*—SRAM, Compute-in-memory (CIM), AdderNet Acceleration, Neural Network (NN) inference.**

I. Introduction

Artificial Intelligence (AI) has experienced significant growth in recent years, emerging as a transformative force that fundamentally alters various aspects of modern life, work, and technological interaction [1]. At the forefront of recent AI advancements are Convolutional Neural Networks (CNNs) [2], which empower unprecedented pattern recognition and computer vision capabilities on both complex datasets and real-world applications such as autonomous driving, video surveillance, and medical image analysis [3]. Nevertheless, CNNs [4] are well known for their substantial computational demands, mainly due to their reliance on computationally intensive Multiply-ACcumulate (MAC) operations [5] for input processing and feature extraction. Additionally, CNNs require frequent memory accesses to retrieve weight matrices and intermediate feature maps, resulting in a continuous and bandwidth-intensive data exchange between the memory hierarchy and processing elements.

The "memory bottleneck" of von Neumann-based processors has prompted the emergence of Compute-in-memory (CIM) [6], a novel memory-centric computing paradigm that aims to minimize data movement via in-situ computation within or near the memory. In this way, CIM architecture drastically reduces the processing latency and energy overheads, making them highly attractive for memory-bounded applications like AI acceleration [7]. CIM designs for efficient CNN training and inference have been extensively studied in recent years across diverse memory technologies [8]–[10]. Memory bitcells and architecture are devised to support matrix-vector-multiplications (MVMs) [11], exploiting the massively-parallelism in memory through analog- and/or digital-domain computations to achieve high-throughput and energy-efficient AI processing. Nonetheless, CNN-CIM still suffers from the inherent complexity of MAC operations, which requires multi-bit multiplications that increase circuit design complexity and power consumption.

Alternatively, aggressive quantized models, such as Binary Neural Networks (BNNs), have been introduced to relax the hardware overheads [12]–[16]. However, BNNs suffer from significant model accuracy loss compared with the CNN counterpart.

Recently, AdderNet [17] has emerged as an efficient alternative to CNNs for efficient neural network inference on resource-limited platforms. Unlike the costly MAC operations in CNNs, AdderNet adopts ℓ_1-norm distance [18] between the input features and weight kernels as the similarity measure, i.e., the absolute difference between the input features and weights. Hence, the expensive MAC operations in CNNs are substituted with efficient sum-of-absolute-difference (SAD) operations in AdderNet. Notably, AdderNet possesses great potential in reducing the expensive MAC operations and power consumption, while simultaneously retaining competitive model accuracies compared with the CNN counterpart across various popular DNN models. Lightweight and energy-efficient AdderNet accelerators have been demonstrated on ASIC- [19]–[21] and FPGA-based [22]–[25] platforms.

In particular, the shift from MAC to SAD processing aligns well with CIM-based DNN accelerator architecture, as this significantly simplifies the bitcell and array designs for implementing in-memory multiplication, as well as mitigating the need of costly near-memory accumulators necessary in

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

CNNs. CIM designs harnessing the AdderNet benefits have been demonstrated with hardware and energy benefits in previous literature [26]–[29]. Nevertheless, similar to the CNN counterpart, AdderNet exhibits a high degree of bit-level and word-level sparsity in both weights and activations. Despite encouraging, existing CIM accelerators do not well exploit the abundant model sparsity in AdderNet as the mathematical formation of SAD function makes it more challenging to exploit such inherent sparsity in AdderNet models for further performance and energy optimization.

In this paper, we present JSA-CIM, a Joint-Sparse AdderNet Compute-in-Memory accelerator for efficient bit-serial acceleration of quantized AdderNet. To the authors' knowledge, this is the first sparsity-aware CIM accelerator explicitly designed for AdderNet, exploiting joint activation and weight sparsity for further energy optimization. The proposed JSA-CIM demonstrates a peak throughput of 170.7 GOPS, 1.8 TOPS/mm^2 area efficiency, and 75.98 TOPS/W energy efficiency, in a 65nm commercial CMOS technology. The main contributions of this paper are summarized as follows:

- We propose a bit-serial, Joint-Sparse AdderNet digital CIM design to allow the exploitation of both bit- and word-level sparsity to minimize the energy consumption,
- We present a novel Joint-Sparse 8T (JS-8T) SRAM design featuring an activation-sparsity-aware read bitline architecture to optimize the dynamic power consumption from reduced read bitline switching, exploiting both bit- and word-level sparsity of weights and activations, respectively,
- We demonstrate a robust, sparsity-gated bit-serial minimum selection circuit that can harness the word-level activation sparsity for substantial power reduction, and completely eliminates the charge-sharing issue that may cause computation errors,
- We incorporate a kernel-aware CIM array organization that well accommodate mainstream kernel sizes to improve the application-level throughput, thereby boosting throughput and area efficiency metrics.

The remainder of this paper is organized as follows. Section II covers the background and motivation. Section III details the JSA-CIM Architecture. Section IV presents the joint-sparse 8T SRAM design with sparsity-aware read bitline architecture. Section V introduces robust and sparsity-gated minimum selection circuit (AdderNet Compute Unit (ACU)) Section VI demonstrates the experimental results and Section VII concludes this paper.

II. Background and Motivation

A. Background of CNN and AdderNet

CNNs rely on convolutional kernels for efficient feature extraction, in which the underlying computation are MAC operations between the feature maps and weight kernels. Convolution in a CNN can be described as follows: given a 4-D weight tensor, where d is the kernel size, c_i is the number of input channels, and c_o is the number of output channels, the output feature map Y is computed as:

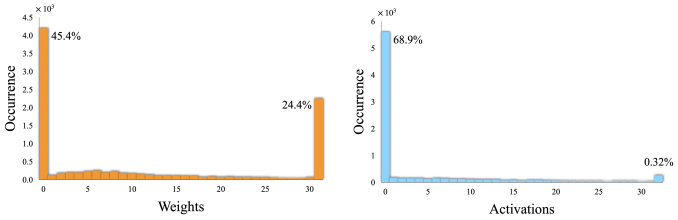


Fig. 1: A representative AdderNet model sparsity illustration.

$$Y_{c_o,h,w} = \sum_{i=0}^{d}\sum_{j=0}^{d}\sum_{k=0}^{c_{in}} F_{i,j,k} \cdot X_{i,h+j,w+k} \tag{1}$$

Nevertheless, MAC operations are known to be computational expensive, hardware demanding, and power hungry. To overcome the inefficiencies of MAC operations, AdderNet introduces a sum-of-absolute-difference (SAD) kernel as an efficient alternative to the prevailing MAC operations:

$$Y_{c_o,h,w} = -\sum_{i=0}^{d}\sum_{j=0}^{d}\sum_{k=0}^{c_{in}} |F_{i,j,k} - X_{i,h+j,w+k}| \tag{2}$$

Comparing Eq. (1) and (2), it is clear that AdderNet is hardware-friendly and can lead to substantial hardware and energy savings as compared with the CNN counterpart.

B. CIM Acceleration for CNN and AdderNet

Extensive CIM research has focused on CNN accelerations [30]–[32], aiming to map dense MAC operations onto compute-capable memory macros and bitcells supporting in-memory multipliers and near-memory accumulators. Such in-memory MAC operations have been widely explored based on various memory technologies. The MAC computations can be accomplished in either analog or digital domains, with their respective strengths and weaknesses. Moreover, CIM designs exploiting sparsity in CNNs have also been widely studied [33]–[35]. CNNs often exhibit a high degree of sparsity in both weights and activations. A sparse input pair (i.e. either the input weight or activation is zero) to the MAC suggests that such computation has no contribution to the final accumulation, thereby computation skipping becomes feasible and yielding improved performance and energy consumption in CIM.

Several CIM designs specifically for AdderNet acceleration have been proposed to leverage the efficient SAD operations [26]–[29]. An absolute-difference-accumulation (ADA) CIM processor based on SRAM was proposed in [26] to achieve bit-parallel SAD operations, where near-memory adders with 3-bit sign prediction circuits were designed to achieve bit-parallel SAD operation between 16/8-bit weights/activations stored in the same memory macro. In [27], a weight-stationary, bit-parallel AdderNet CIM "iMAD" based on Spin-Transfer Torque (STT)-MRAM was proposed for 8-bit quantized AdderNet. Specifically, a dedicated in-memory subtraction circuit was designed to optimize the baseline "NOT +

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

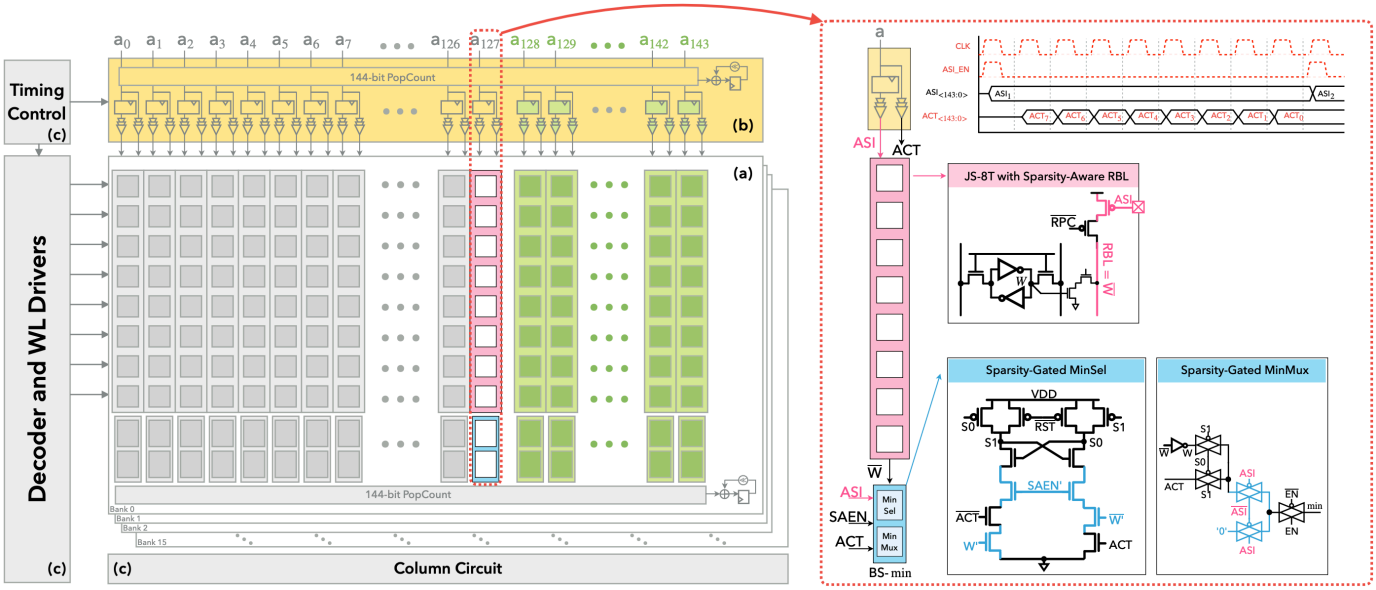


Fig. 2: Joint-Sparse AdderNet CIM Architecture

ADD" scheme to achieve efficient SAD operations to facilitate inference. In [28], a bit-serial weight-stationary CIM was proposed for accelerating AdderNet inference, where SAD operations were transformed into a mathematical equivalence of bit-serial non-negative minimum operation between weight and activation. In [29], a Toggle-Spin-Torque (TST) MRAM-based weight-stationary CIM was introduced for accelerating AdderNet.

C. Challenge and Motivation

The existing literature collectively emphasizes the potential of supporting SAD computation in CIM architecture to enable efficient AdderNet inference. Similar to the CNN counterpart, AdderNet also exhibits inherent model sparsity, as illustrated in Fig. 1. Although the exploitation of sparsity in CIM-based CNN accelerators has been extensively studied, the investigation of AdderNet sparsity in the context of CIM remains limited. This is primarily due to the fundamental differences in computational paradigms. Unlike MAC operations, where zero weights or activations can be readily utilized to skip computations, the SAD operation does not inherently benefit from such sparsity, thereby posing significant challenges for its exploitation. The primary motivation of this work is to address this limitation by exploring joint sparsity in both weights and activations of AdderNet, aiming to enhance the energy efficiency of CIM-based inference accelerators tailored for AdderNet.

III. Joint-Sparse AdderNet CIM Architecture

In this section, we detail the architecture design of JSA-CIM to support joint-sparsity exploitation originated from both weights and activations.

A. SAD Transformation for Bit-Serial Sparsity Exploitation

The baseline mathematical formulation of SAD hinders the opportunistic sparsity exploitation. In JSA-CIM, we resort to an equivalent transformation of SAD introduced in [28]:

$$\sum |x_i - w_i| \equiv \sum x_i + \sum |w_{i,n}| - 2\sum \min(x_i, \mathrm{ReLU}(w_i)) \quad (3)$$

Eq. (3) suggests that the SAD kernel in AdderNet can be obtained via on-the-fly activation accumulation, pre-compute the sum of the absolute negative weights, and the sum of the minimum between the input activations and "ReLU-ed" weights. Hence, the third term in Eq. (3), i.e., the sum of the minimum between the inputs activations and the weights, will be mapped and computed in parallel in CIM. Additionally, Eq. (3) is universal for both floating and integer data representation, thereby well supporting quantized AdderNet.

B. Architecture Support for Joint-Sparsity Exploitation

The overall architecture of JSA-CIM is illustrated in Fig. 2. While our design is based on the foundational CIM architecture presented in [28], it incorporates several notable modifications aimed sparsity exploitation and robustness enhancement for CIM AdderNet inference. JSA-CIM is mainly composed of: (a) a Joint-Sparse AdderNet Compute Macro (JSACM), (b) an Activation-Handling Unit (AHU), and (c) other peripheral circuitry including row decoders, wordline drivers, and timing control unit, etc.

1) Joint-Sparse AdderNet Compute Macro: JSA-CIM features a bit-serial, weight-stationary, joint-sparse AdderNet Compute Macro based on an 8T-SRAM bitcell. The CIM array is constructed with 16 banks, and each bank is designed to support 144 columns of 8-bit column-wise weight storage, as opposed to 128 columns in [28]. Model weights are stored in a column-wise fashion to facilitate massively-parallel bit-serial minimum (BS- min) computation outlined in Eq. (3).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

The addition of columns indeed increase the array size by 12.5%, however, this design choice enables the CIM array to handle mainstream DNN kernels (e.g., 3×3 or 7×7) more efficiently as it avoids the iterative weight loading process, ultimately leading to improved throughput and throughput-per-area metrics.

To support Joint-Sparsity exploitation in the AdderNet Compute Macro, we harness the data-dependent 8T SRAM bitcell and devise the read bitline architecture to explore bit-level weight sparsity and word-level activation sparsity to minimize RBL precharge and discharge activities. Moreover, we modify the near-memory Bit-Serial Minimum (BS-min) circuit to allow word-level activation sparsity-gated BS-min for dynamic power reduction. The circuit details of the JS-8T with Sparsity-Aware RBL design, and the Sparsity-Gated BS-min will be discussed in Section IV and V, respectively.

2) Activation Handling Unit: Similar to other weight-stationary CIM designs, JSA-CIM allows to maximally share and re-use the input activations loaded from external activation memories. To ensure correct SAD operation according to Eq. (3) and simultaneously exploit word-level activation sparsity, a 144-bit Activation-Handling Unit (AHU) was designed to support on-the-fly activation accumulation via popcount and shift-accumulate circuit, a latch array to store the word-level Activation Sparsity Indicator (ASI) computed from previous layers, and activation buffers driving 16 banks in this work.

C. Activation-Sparsity-Aware Bit-Serial Dataflow

With the circuit modifications in JSA-CIM described earlier, we devise the bit-serial dataflow to allow word-level activation sparsity exploitation. For a baseline bit-serial CIM design for AdderNet [28], we assume an operating frequency of f-MHz for 8-bit AdderNet inference. The BS-min computation was achieved via two main circuit component: (i) a MSB-first minimum selection (`MinSel`) unit, and (ii) a minimum multiplexer (`MinMux`) to output the bit-wise minimum. In each cycle, a pair of one-bit weight and activation is sent to `MinSel` to generate a control signal for the `MinMux` to output the minimum to the final popcount circuit. The partial sum of 1-bit minimum across all columns will be shifted and added in the following cycles to obtain the sum of minimum $\sum \min(x_i, w_i)$. Hence, a total of 8 cycles are required for one input activation.

To exploit the word-level activation sparsity, an Activation Sparsity Indicator (ASI) is required to be loaded into JSA-CIM. ASI provides column-wise granularity control by the activation sparsity, which will be used to determine whether a weight readout or a bit-wise comparison can be skipped to save dynamic power consumption associated with the array read bitlines and bit-serial minimum (BS-min) operation.

Note that this activation sparsity indicator, ASI, will be loaded in the first cycle preceding the normal 8-cycle BS-min computation. Hence, JSA-CIM accomplishes sparsity-aware BS-min computation in 9 cycles. To achieve the same throughput, JSA-CIM should be designed to be operated at $1.125f$-MHz. We will demonstrate that, JSA-CIM can still yield better energy-efficiency by exploiting the sparsity, despite of 12.5% higher operating frequency.

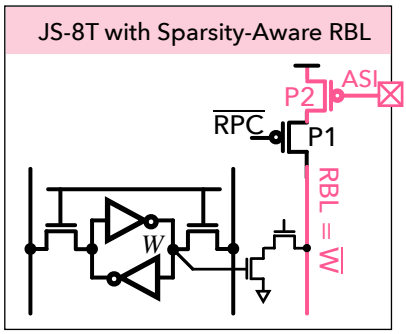


Fig. 3: Joint-Sparse 8T SRAM with Sparsity-Aware RBL controlled by column-wise sparsity indicator ASI.

TABLE I: JS-8T RBL Precharge/Discharge Behaviors

ASI	Weight	Regular 8T		Joint-Sparsity 8T	
		Pre-C	Dis-C	Pre-C	Dis-C
0	0	✓	✗	✓	✗
	1	✓	✓	✓	✓
1	0	✓	✗	✗	✗
	1	✓	✓	✗	✓

IV. JS-8T SRAM with Sparsity-Aware RBL Architecture

JSA-CIM exploits word-level activation sparsity whenever the 8-bit activation is '0', which presents the opportunity to skip the weight readout from the SRAM bitcell, by preventing the bitlines from being precharged. As mentioned earlier, ASI will be loaded to indicate whether the 8-bit activation word is '0' in a fine-grained column-wise fashion. In the case of the 8-bit activation word being '0', we set ASI bit to be '1'. Otherwise, ASI bit will be set to '0'.

Fig. 3 depicts the joint-sparse 8T design with sparsity-aware read bitline architecture. In addition to the conventional RBL precharge transistor P1 controlled by $\overline{\text{RPC}}$, one additional PMOS transistor P2 controlled by the sparsity indicator ASI is stacked in the precharge path. P2 can, therefore, clamp the precharge when a zero activation ($ASI = 1$) is asserted, leading to almost no RBL dynamic power consumption. On the other hand, when the activation is not sparse ($ASI = 0$), then the P1-P2 stack can still provide normal precharge functionality. To ensure the same precharge performance, P1 and P2 transistors are required to be upsized accordingly.

Moreover, JSA-CIM can still harness the bit-level sparsity inherent to the 8T SRAM when non-zero activation is present ($ASI = 0$). In an 8T SRAM bitcell, whenever a '1' is stored, a read operation will not discharge the bitline and ultimately does not incur dynamic power consumption over RBL. Such data-dependent behavior of 8T SRAM has been previously explored to save read energy [36] [37], whereas JSA-CIM allows joint-sparsity exploitation with the sparsity-aware RBL design of 8T SRAM. In JSA-CIM, the read port is connected to internal node W ($\overline{Q}$) to utilize the weight sparsity.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

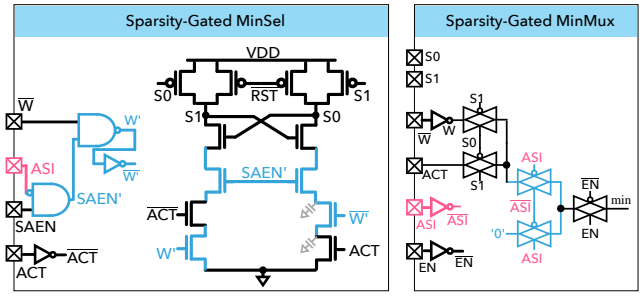


Fig. 4: Schematic illustration of the proposed Sparsity-Gated `MinSel` Unit, and the Sparsity-Gated `MinMux`.

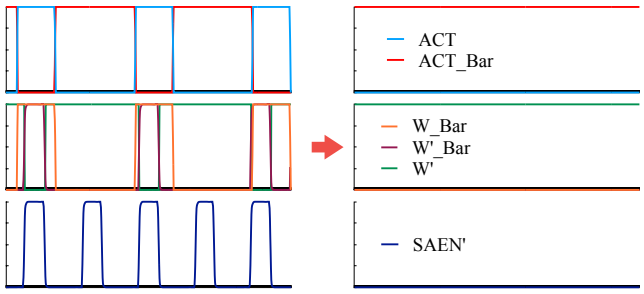


Fig. 5: Timing diagram to illustrate the baseline `MinSel` input signals without exploiting sparsity (left column), and Sparsity-Gated input signals (right column).

Table I summarizes the RBL behaviors of both precharge (Pre-C) and discharge (Dis-C) of the read bitline depending on the status of the ASI and stored weight in the Joint-Sparse 8T SRAM design.

V. Robust and Sparsity-Gated Minimum Selection Circuitry

JSA-CIM further employs ASI to minimize the dynamic power consumption of the BS-min operation required in $\sum \min(x_i, w_i)$, as illustrated in Fig. 4. In particular, the minimum selection circuit contributes to a substantial amount of dynamic power, which is mainly caused by the excessive input switching during bit-serial minimum selection. Moreover, the `MinSel` unit in [28] adopts a dynamic-logic-based comparator to achieve bit-serial comparison and issues a proper control to the subsequent `MiMux` to output the minimum between weight and activation. However, this design suffers from a voltage droop on the output nodes due to charge-sharing, potentially error-prone. A more robust design is therefore highly desired.

Our proposed robust and sparsity-aware BS-min consists of three key components: (1) a sparsity-gated `MinSel` unit, (2) a sparsity-gated minimum multiplexer `MinMux`, and (3) the minimum accumulator `MinAcc`. Fig. 4 shows the improved design of the robust and sparsity-gated `MinSel` unit. In summary, word-level activation sparsity is used to minimize unnecessary bit-serial input switching to save dynamic power, and meanwhile eliminate potential charge-sharing issues.

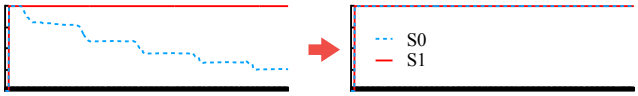


Fig. 6: Timing diagram of output S0 voltage droop issue (left column). The gated weight signals completely eliminate the charge-sharing issue in `MinSel` (right column).

TABLE II: `MinSel` & `MinMux` Truth Table.

W	ACT	ASI	S0	S1	Min
0	0	0	1	1	NV
0	1	0	1	0	W
1	0	0	0	1	ACT
1	1	0	1	1	NV
DC	DC	1	NV	NV	'0'

DC = Don't Care, NV = Not Valid

SAEN & Weight Gating: Our proposed sparsity-gated `MinSel` exploits the word-level activation sparsity to reduce the dynamic power consumption by preventing the excessive bit-serial switching activities, as illustrated in Fig. 4. An '`AND`' gate is used to gate the $SAEN$ signal with ASI. When $ASI = 1$, the sparsity-gated signal $SAEN'$ will be turned off, thereby preventing the `MinSel` from dynamic power consumption. A second '`AND`' gate controlled by $SAEN'$ is introduced to avoid the unnecessary weight switching during RBL precharge. Consequently, the bit-serial input W' and $\overline{W'}$ to the `MinSel` unit are gated to save dynamic power. Fig. 5 illustrates the switching profiles of the `MinSel` inputs with or without the sparsity-gated feature. As can be observed, employing the sparsity-gating (right) ensures negligible input switching for significant amount of dynamic power saving, as opposed to the design without sparsity gating (left).

Moreover, the weight-gating over the differential inputs ensures charge-sharing-free operation, thereby leading to no voltage droop on the output node. Fig. 6 (left) illustrates the worst-case scenario where consecutive '1's in both activations and weights ($ACT = 11111$, $W = 11111$) cause errors in previous `MinSel` design due to charge-sharing issue (i.e., S0 discharged from '1' to '0', indicating a wrong minimum selection control), whereas the improved weight-gating design in this work (right) completely eliminates this issue between the output node and the parasitic diffusion capacitance. The truth table of the proposed sparsity-gated `MinSel` circuit, together with the `MinMux` logic, is summarized in Table II.

Finally, the count of '1's in the output of MinMux is summed up in a 144b-wide pop-count, and afterwards shifted to left (multiply by 2) yielding the needed $2\sum \min(x_i, \text{ReLU}(w_i))$ computation result to complete the aforementioned SAD function.

VI. Experimental Results

The proposed JSA-CIM was implemented in a commercial 65-nm CMOS technology. JSA-CIM has a total weight storage capacity of 18-kbit and occupies a layout area of $0.0948\ mm^2$ ($303\mu m \times 313\mu m$), as illustrated in Fig. 7. It consists of 16 banks of AdderNet Compute Macro. In each bank, a 144-bit

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

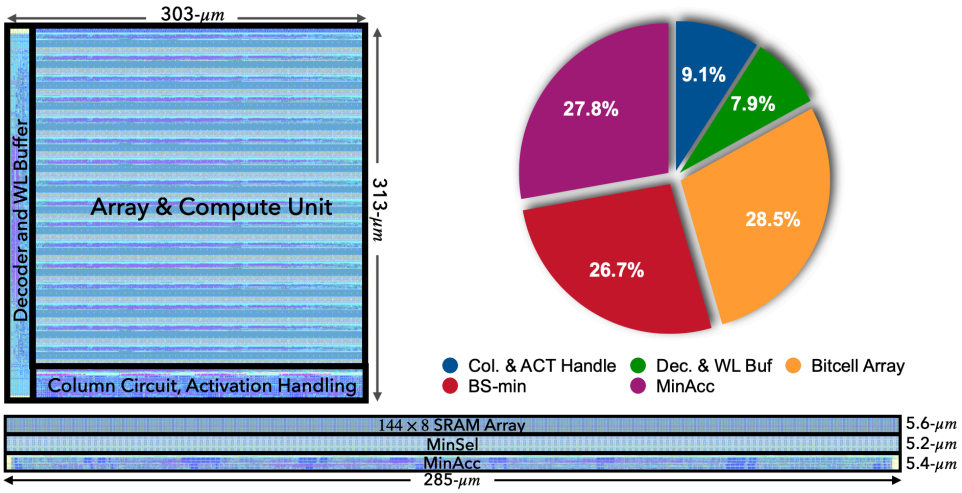


Fig. 7: Annotated Layout of JSA-CIM design (top left), area breakdown (top right), and the AdderNet Compute Unit (bottom).

$\times$ 8-row 8T SRAM array is designed to support mainstream kernel sizes in DNN models. The area breakdown of JSA-CIM is shown in Fig. 7. ACU occupies around 54.5% of the whole JSA-CIM macro, while the SRAM array and column & activation handling circuit take approximately 37.6% of the macro. With a nominal supply voltage of 1V and room temperature of 27°C, JSA-CIM can be reliably operated up to 333 MHz across all process corners. With this frequency, JSA-CIM achieves a peak throughput of 170.7-GOPS for 8-bit AdderNet model.

A. Joint-Sparsity Energy Savings

Fig. 8 (a-b) demonstrate the normalized energy savings from the proposed Joint-Sparsity Exploitation. As can be observed in Fig. 8 (a), significant energy savings can be observed from reduced amount of precharge energy of RBL in the SRAM array, mainly contributed by the joint weight bit-level and word-level activation sparsity. Previous work [28] passively harnesses pure bit-level weight sparsity, which leads to 57% energy reduction, while joint-sparsity in our work achieves up to 76% energy reduction.

Furthermore, in Fig. 8 (b), the inclusion of word-level activation sparsity allows substantial energy reduction for the `MinSel` unit, as opposed to previous literature [28]. JSA-CIM leads to 48% energy reduction in `MinSel`, from the reduction of the excessive switching activities from weights and activations and completely shutting off the circuit whenever $ASI = 1$. Note that the aforementioned energy benefits have already included the energy overheads of the extra cycle for loading ASI, demonstrating the effectiveness of joint-sparsity exploitation.

Streaming in the additional ASI to be able to exploit ACT sparsity has presented a 12.5% power consumption overhead, however, the implemented joint-sparsity scheme has compensated for the increase in frequency to offset the power consumption overhead. Fig. 8(c-d) show the additional power consumed of running at 333 MHz as opposed to 296 MHz, and how JSA-CIM has recouped such overhead in addition to substantial extra power savings. JSA-CIM achieves 36.5% and 41% power saving of bitline precharge and `MinSel`, respectively compared to running at 296 MHz with only weight sparsity being exploited.

TABLE III: Application-level (with a kernel size of 3×3) throughput comparison of JSA-CIM with previous work.

	Array Throughput	SAD Latency	Application Throughput	Energy Efficiency
JSSC [28] @ 240 MHz	122 GOPS	33.3ns (8 cycles)	61 GOPS	51 TOPS/W
This Work @ 270 MHz	137 GOPS	33.3ns (9 cycles)	137 GOPS	123 TOPS/W

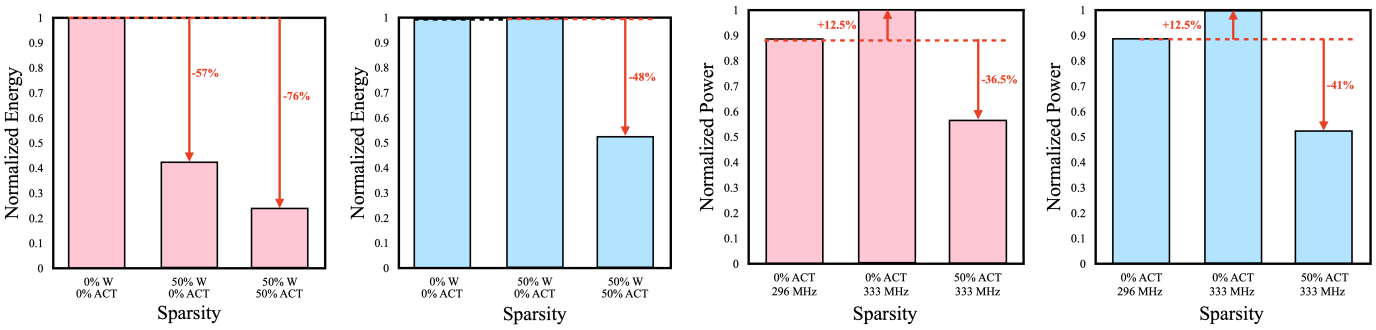
B. Application-Level Validation

JSA-CIM employs a kernel-size friendly, 144-bit-wide architecture to accommodate AdderNet inference with popular kernel sizes. For instance, 144-bit JSA-CIM well supports 3×3 kernel without excessive weight loading, whereas 128-bit organization in [28] leads to halved effective throughput. Table III highlights the quantitative benefits of JSA-CIM over [28]. In the prior design, throughput and energy efficiency are effectively halved when using a 3x3 kernel, yielding 61 GOPS and 51 TOPS/W instead of the reported 122 GOPS and 102 TOPS/W, respectively. Although JSA-CIM incurs a column number increase and operates at a higher frequency, the effective throughput and energy efficiency are significantly improved (2.2× and 2.4×) compared to previous work.

C. Comparison

Table IV depicts the comparison between this work and the state-of-the-art CIM works. In comparison with other AdderNet CIM designs, this is the first work to exploit both activation and weight sparsity, which reflects clear energy savings benefits compared to other CIM macros achieving around 20% higher normalized energy efficiency with respect to the best previous work, while maintaining and assuring the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.



(a) Energy savings of JS-8T at 333 MHz.

(b) Energy savings of `MinSel` at 333 MHz.

(c) JS-8T Power consumption at fixed 50% weight sparsity.

(d) `MinSel` Power consumption at fixed 50% weight sparsity.

Fig. 8: Energy and power consumption trends of JS-8T and `MinSel` across different frequencies and sparsity levels of both activation and weights.

TABLE IV: Comparison with The State-of-the-Art Works

	ISSCC'22 P. Wu [38]	ISSCC'22 H. Fujiwara [39]	ISSCC'22 B. Yan [30]	TCAS-II'21 S. Um [26]	JSSC'25 H. Diao [28]	TVLSI'25 Z. Qian [40]	This Work
Technology	28nm	5nm	28nm	28nm	28nm	28nm	**65nm**
Memory	SRAM	SRAM	SRAM	SRAM	SRAM	RRAM	**SRAM**
CIM Type	Analog	Digital	Digital	Digital	Digital	Digital	**Digital**
NN Function	MAC	MAC	MAC	SAD	ℓ_1-distance	ℓ_1-distance	**ℓ_1-distance**
Array Size	1Mb	64Kb	32Kb	230.13KB	16Kb	32Kb	**18Kb**
Bitcell	Compact 6T	12T	Compact 6T	6T	Compact 8T	1T1R	**8T**
Macro Area (mm^2)	N/A	0.0133	0.03	3.78	0.028	0.038	**0.0947**
Input Bit	4/8b	4b	1-8b	8b	2-8b	8b	**8b**
Weight Bit	4/8b	4b	1/4/8b	16b	2-8b	8b	**8b**
Area Efficiency (TOPS/mm^2)	N/A	13.8@(0.5V)	0.18@(0.8V)	0.019@(1V)	4.4@(0.9V)	12.8@(0.9V)	**1.8@(1V)**
Energy Efficiency (TOPS/W)	37.01	63@(0.5V)	27.38@(0.8V)	43.1@(1V)	102@(0.54V)	126@(0.8V)	**75.98@(1V)**
Normalized Energy Efficiency (TOPS/W)[1]	37.01	10.49	27.38	43.1	102	126	**151.96**

[1]Normalized to $28nm$ technology node

stability, robustness and correctness of the results. Moreover, due to JSA-CIM's 144-bit wide architecture, it maximizes the effective throughput by up to 2× compared to [28] depending on the kernel size.

VII. Conclusion

Overcoming the von Neumann computation bottleneck, and bypassing the expensive and complicated multiplication operation; this paper presented JSA-CIM, an 8T-SRAM-based digital compute-in-memory (CIM) macro designed for energy-efficient AdderNet acceleration, implemented in 65nm CMOS technology. By leveraging a 144-bit-wide, kernel size-aware organization, JSA-CIM maximizes throughput for weight-stationary AdderNet across multiple kernel sizes.

JSA-CIM introduces a joint-sparsity-aware design that exploits sparsity in both activations and weights. The novel sparsity-aware precharge circuit design uses ASI-controlled clamping transistors to dynamically prevent unnecessary bitline pre-charging, significantly reducing energy consumption. Additionally, the modified 8T-SRAM bitcell leverages weight sparsity to minimize bitline discharging, further enhancing energy efficiency. Another key innovation in JSA-CIM is the improved `MinSel` design in the ACU, which ensures stable and accurate minimum control signal generation while yielding further energy savings. Experiments on JSA-CIM's performance, running at 333 MHz, have demonstrated 75.98 TOPS/W energy efficiency, 170.7 GOPS throughput, and 1.8 TOPS/mm^2 area efficiency. Moreover, JSA-CIM's architecture enables high application-level throughput across popular kernel sizes.

Acknowledgment

This work was in part supported by the U.S. Department of Agriculture NACA contract 58-8042-3-047, in part by the funding from Integrated Electronics Engineering Center (IEEC) at Binghamton University, New York State Division of Science, Technology and Innovation, and in part by the Binghamton University Transdisciplinary Areas of Excellence (TAEs) Data Science grant.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.

[2] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] F.-F. Li, J. Deng, J. Platt *et al.*, "Artificial intelligence: A modern approach to real-world applications," *Communications of the ACM*, vol. 64, no. 6, pp. 36–38, 2021.

[4] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[5] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, vol. 28, 2015.

[6] H. S. Stone, "A logic-in-memory computer," *IEEE Transactions on Computers*, vol. C-19, no. 1, pp. 73–78, 1970.

[7] X. Zou, S. Xu, X. Chen, L. Yan, and Y. Han, "Breaking the von neumann bottleneck: architecture-level processing-in-memory technology," *Science China Information Sciences*, vol. 64, no. 6, p. 160404, 2021.

[8] Y. LeCun, B. Boser, J. S. Denker, D. Henderson *et al.*, "Backpropagation applied to handwritten zip code recognition," *Neural computation*, vol. 1, no. 4, pp. 541–551, 1989.

[9] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian *et al.*, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 14–26, 2016.

[10] S. Hamdioui, L. Xie, H. A. Du Nguyen, M. Taouil *et al.*, "Memristor based computation-in-memory architecture for data-intensive applications," in *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2015, pp. 1718–1725.

[11] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969.

[12] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "Xnor-sram: In-memory computing sram macro for binary/ternary deep neural networks," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, 2020.

[13] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1," *arXiv preprint arXiv:1602.02830*, 2016.

[14] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: Training neural networks with low precision weights and activations," *journal of machine learning research*, vol. 18, no. 187, pp. 1–30, 2018.

[15] P. Katti, C. Ruah, O. Simeone, B. M. Al-Hashimi, and B. Rajendran, "Bayes2imc: In-memory computing for bayesian binary neural networks," *arXiv preprint arXiv:2411.07902*, 2024.

[16] M.-S. Le, T.-N. Pham, T.-D. Nguyen, and I.-J. Chang, "Pr-cim: A variation-aware binary-neural-network framework for process-resilient computation-in-memory," *arXiv preprint arXiv:2110.09962*, 2021.

[17] H. Chen, Y. Wang, C. Xu, B. Shi *et al.*, "Addernet: Do we really need multiplications in deep learning?" in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 1465–1474.

[18] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996. [Online]. Available: http://www.jstor.org/stable/2346178

[19] D. T. Nguyen, D. T. Vo, D. A. Ngo, and K. C. Kim, "Addernet based cnn accelerator with asic implementation for edge ai," in *2021 IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*. IEEE, 2021, pp. 1–2.

[20] Y. Wang, M. Huang, K. Han, H. Chen *et al.*, "Addernet and its minimalist hardware design for energy-efficient artificial intelligence. arxiv 2021," *arXiv preprint arXiv:2101.10015*, 2021.

[21] M. Zhuang, X. Liao, H. Wu, J. Zhou, and Z. Guo, "Vlsi architecture design for adder convolution neural network accelerator," in *2021 IEEE 15th International Conference on Anti-counterfeiting, Security, and Identification (ASID)*. IEEE, 2021, pp. 5–9.

[22] Y. Zhang, O. A. Kailani, B. Zhou, and W. Zhao, "Addernet 2.0: Optimal addernet accelerator designs with activation-oriented quantization and fused bias removal-based memory optimization," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–11, 2025.

[23] Y. Zhang, B. Sun, W. Jiang, Y. Ha, M. Hu, and W. Zhao, "Wsq-addernet: Efficient weight standardization based quantized addernet fpga accelerator design with high-density int8 dsp-lut co-packing optimization," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 2022, pp. 1–9.

[24] J. Faraone, M. Kumm, M. Hardieck, P. Zipf *et al.*, "Addnet: Deep neural networks using fpga-optimized multipliers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 1, pp. 115–128, 2019.

[25] N. Zhang, S. Ni, L. Chen, T. Wang, and H. Chen, "High-throughput and energy-efficient fpga-based accelerator for all adder neural networks," *IEEE Internet of Things Journal*, 2025.

[26] S. Um, S. Kim, S. Kim, and H.-J. Yoo, "A 43.1tops/w energy-efficient absolute-difference-accumulation operation computing-in-memory with computation reuse," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 5, pp. 1605–1609, 2021.

[27] S. Zhu, S. Li, and W. Liu, "imad: An in-memory accelerator for addernet with efficient 8-bit addition and subtraction operations," in *Proceedings of the Great Lakes Symposium on VLSI 2022*, ser. GLSVLSI '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 65–70. [Online]. Available: https://doi.org/10.1145/3526241.3530313

[28] H. Diao, Y. He, X. Li, C. Tang *et al.*, "A multiply-less approximate sram compute-in-memory macro for neural-network inference," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 2, pp. 695–706, 2025.

[29] L. Luo, E. Deng, D. Liu, Z. Wang *et al.*, "Citst-addernets: Computing in toggle spin torques mram for energy-efficient addernets," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 3, pp. 1130–1143, 2024.

[30] B. Yan, J.-L. Hsu, P.-C. Yu, C.-C. Lee *et al.*, "A 1.041-mb/mm2 27.38-tops/w signed-int8 dynamic-logic-based adc-less sram compute-in-memory macro in 28nm with reconfigurable bitwise operation for ai and embedded applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 188–190.

[31] A. Kneip, M. Lefebvre, P. Maistriaux, and D. Bol, "Imagine: An 8-to-1b 22nm fd-soi compute-in-memory cnn accelerator with an end-to-end analog charge-based 0.15-8pops/w macro featuring distribution-aware data reshaping," *arXiv preprint arXiv:2412.19750*, 2024.

[32] M. Li, H. Zhu, S. He, H. Zhang *et al.*, "Slam-cim: A visual slam backend processor with dynamic-range-driven-skipping linear-solving fp-cim macros," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 11, pp. 3853–3865, 2024.

[33] W. Zhang, S. Ando, Y.-C. Chen, S. Miyagi, S. Takamaeda-Yamazaki, and K. Yoshioka, "Pacim: A sparsity-centric hybrid compute-in-memory architecture via probabilistic approximation," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3676536.3676704

[34] A. Sridharan, F. Zhang, J.-S. Seo, and D. Fan, "Sp-imc: A sparsity aware in-memory-computing macro in 28nm cmos with configurable sparse representation for highly sparse dnn workloads," in *2024 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 2024, pp. 1–2.

[35] S. Kim, S. Kim, S. Hong, S. Kim, D. Han, and H.-J. Yoo, "C-dnn: A 24.5-85.8 tops/w complementary-deep-neural-network processor with heterogeneous cnn/snn core architecture and forward-gradient-based sparsity generation," in *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2023, pp. 334–336.

[36] J. Chen, W. Zhao, Y. Wang, Y. Shu, W. Jiang, and Y. Ha, "A reliable 8t sram for high-speed searching and logic-in-memory operations," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 6, pp. 769–780, 2022.

[37] Y. Kim, S. Li, N. Yadav, and K. K. Choi, "A novel ultra-low power 8t sram-based compute-in-memory design for binary neural networks," *Electronics*, vol. 10, no. 17, p. 2181, 2021.

[38] P.-C. Wu, J.-W. Su, Y.-L. Chung, L.-Y. Hong *et al.*, "A 28nm 1mb time-domain computing-in-memory 6t-sram macro with a 6.6ns latency, 1241gops and 37.01tops/w for 8b-mac operations for edge-ai devices," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[39] H. Fujiwara, H. Mori, W.-C. Zhao, M.-C. Chuang *et al.*, "A 5-nm 254-tops/w 221-tops/mm2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous mac and write operations," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 1–3.

[40] Z. Qian, S. Yan, Z. Dai, Z. Guo *et al.*, "An rram-based computing-in-memory macro with low-power readout/hold circuits and activation differential strategy for addernet," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, pp. 1–5, 2025.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:46:11 UTC from IEEE Xplore. Restrictions apply.